

Informatics 141 Computer Science 121

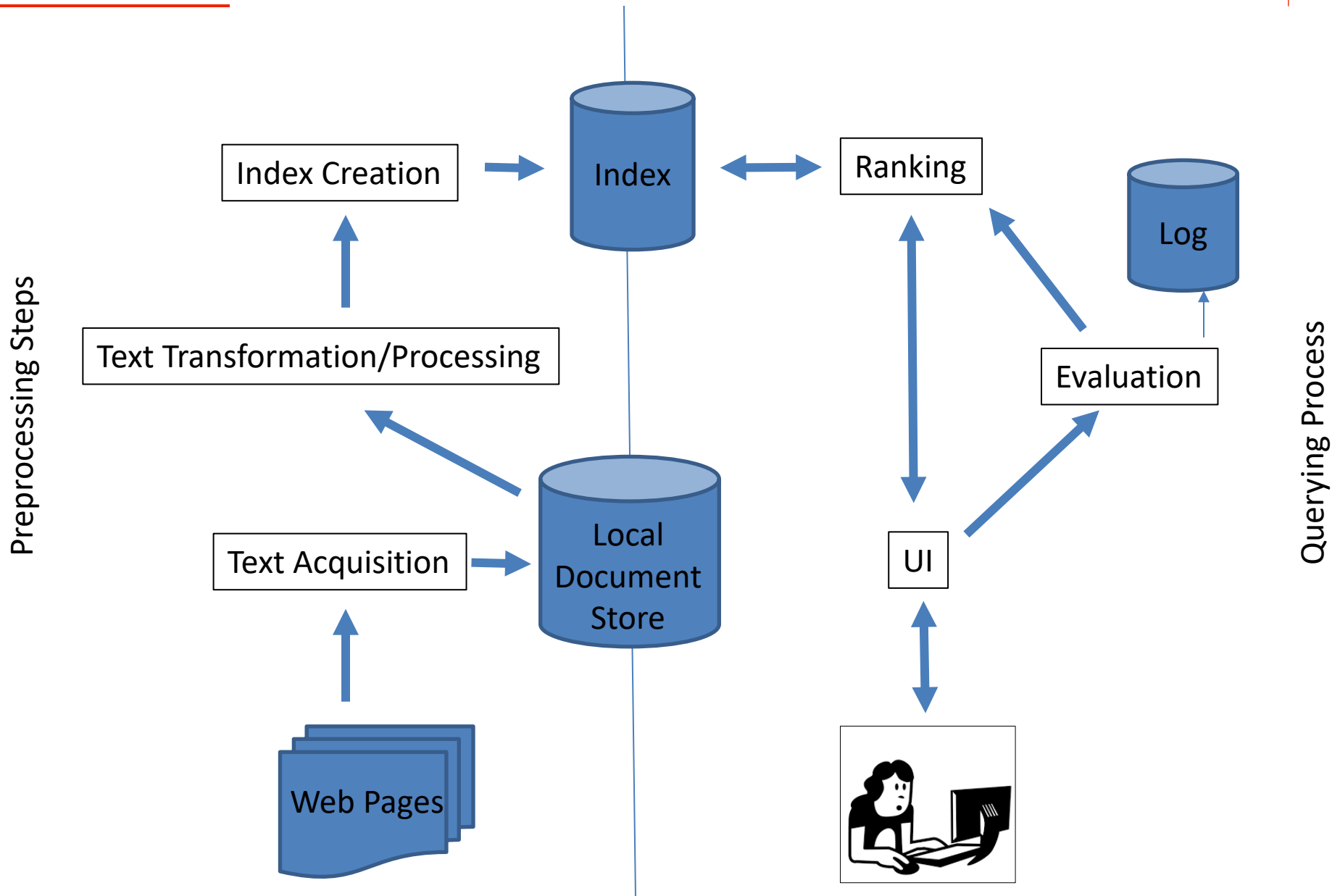
Information Retrieval

Lecture 12

Duplication of course material for any commercial purpose without the explicit written permission of the professor is prohibited.

These course materials borrow, with permission, from those of Prof. Cristina Videira Lopes, Prof. Alberto Krone-Martins, Addison Wesley 2008, Chris Manning, Pandu Nayak, Hinrich Schütze, Heike Adel, Sascha Rothe, Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie. Powerpoint theme by Prof. André van der Hoek.

Architecture



Detecting duplicates and removing noise

Information Retrieval

- Similarity comparisons using word-based representations of a document are more effective at finding near-duplicates than fingerprint techniques
 - Problem is efficiency

- Similarity comparisons using word-based representations of a document are more effective at finding near-duplicates than fingerprint techniques
 - Problem is efficiency
- **Simhash** was introduced by Charikar, 2002 to **combine the advantages of the word-based similarity measures with the efficiency of fingerprints based on hashing**

- Similarity comparisons using word-based representations of a document are more effective at finding near-duplicates than fingerprint techniques
 - Problem is efficiency
- Simhash was introduced by Charikar, 2002 to combine the advantages of the word-based similarity measures with the efficiency of fingerprints based on hashing
 - *Similarity of pages measured by the cosine correlation is proportional to the number of bits that are the same in simhash fingerprints*



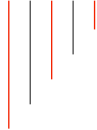
1. Process the document into a set of features with associated weights. We will assume the simple case where the features are words weighted by their frequency.



1. Process the document into a set of features with associated weights. We will assume the simple case where the features are words weighted by their frequency.
2. Generate a hash value with b bits (the desired size of the fingerprint) for each word. The hash value should be unique for each word.



1. Process the document into a set of features with associated weights. We will assume the simple case where the features are words weighted by their frequency.
2. Generate a hash value with b bits (the desired size of the fingerprint) for each word. The hash value should be unique for each word.
3. In b -dimensional vector V , update the components of the vector by adding the weight for a word to every component for which the corresponding bit in the word's hash value is 1, and subtracting the weight if the value is 0.



1. Process the document into a set of features with associated weights. We will assume the simple case where the features are words weighted by their frequency.
2. Generate a hash value with b bits (the desired size of the fingerprint) for each word. The hash value should be unique for each word.
3. In b -dimensional vector V , update the components of the vector by adding the weight for a word to every component for which the corresponding bit in the word's hash value is 1, and subtracting the weight if the value is 0.
4. After all words have been processed, generate a b -bit fingerprint by setting the i th bit to 1 if the i th component of V is positive, or 0 otherwise.

Simhash Example

Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.

(a) Original text

Simhash Example

Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.

(a) Original text

tropical 2 fish 2 include 1 found 1 environments 1 around 1 world 1
including 1 both 1 freshwater 1 salt 1 water 1 species 1

(b) Words with weights

Simhash Example

Tropical fish include fish found in tropical environments around the world,
including both freshwater and salt water species.

(a) Original text

tropical 2 fish 2 include 1 found 1 environments 1 around 1 world 1
including 1 both 1 freshwater 1 salt 1 water 1 species 1

(b) Words with weights

tropical	01100001	fish	10101011	include	11100110
found	00011110	environments	00101101	around	10001011
world	00101010	including	11000000	both	10101110
freshwater	00111111	salt	10110101	water	00100101
species	11101110				

(c) 8 bit hash values

Simhash Example

Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.

(a) Original text

tropical 2 fish 2 include 1 found 1 environments 1 around 1 world 1
including 1 both 1 freshwater 1 salt 1 water 1 species 1

(b) Words with weights

	- + + - - - - +		+ - + - + - + +		+ + + - - + + -
tropical	01100001	fish	10101011	include	11100110
found	00011110	environments	00101101	around	10001011
world	00101010	including	11000000	both	10101110
freshwater	00111111	salt	10110101	water	00100101
species	11101110				

(c) 8 bit hash values

1
-5
9
-9
3
1
3
3

3
=
2
+
2
-
1
-
1
+
1
+
1
-
1
-
1
-
1
+
1
+
1
+
1
-
1

(d) Vector *V* formed by summing weights

Simhash Example

Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.

(a) Original text

tropical 2 fish 2 include 1 found 1 environments 1 around 1 world 1
including 1 both 1 freshwater 1 salt 1 water 1 species 1

(b) Words with weights

tropical	01100001	fish	10101011	include	11100110
found	00011110	environments	00101101	around	10001011
world	00101010	including	11000000	both	10101110
freshwater	00111111	salt	10110101	water	00100101
species	11101110				

(c) 8 bit hash values

1 -5 9 -9 3 1 3 3

(d) Vector V formed by summing weights

1 0 1 0 1 1 1 1

(e) 8-bit fingerprint formed from V

1. Process the document into a set of features with associated weights. We will assume the simple case where the features are words weighted by their frequency.
2. Generate a hash value with b bits (the desired size of the fingerprint) for each word. The hash value should be unique for each word.
3. In b -dimensional vector V , update the components of the vector by adding the weight for a word to every component for which the corresponding bit in the word's hash value is 1, and subtracting the weight if the value is 0.
4. After all words have been processed, generate a b -bit fingerprint by setting the i th bit to 1 if the i th component of V is positive, or 0 otherwise.

- So, how similar are two text files A and B?
 - Compute the simhashes H_A and H_B for each text.
 - The similarity is simply the fraction of the bits that are the same over all n bits of the representation.

$$S_{A,B} = \frac{\sum_n (\mathcal{H}_A \equiv_{\text{bitwise}} \mathcal{H}_B)}{n}$$

- So, how similar are two text files A and B?
 - Compute the simhashes H_A and H_B for each text.
 - The similarity is simply the fraction of the bits that are the same over all n bits of the representation.

$$S_{A,B} = \frac{\sum_n (\mathcal{H}_A \equiv_{\text{bitwise}} \mathcal{H}_B)}{n}$$

- Are text A and text B near duplicates?
 - Define a threshold level τ .
 - If similarity is greater or equal than the threshold, they are near duplicates **under your definition of threshold**.

$$S_{A,B} \geq \tau \implies A \text{ \& B are near duplicates}$$

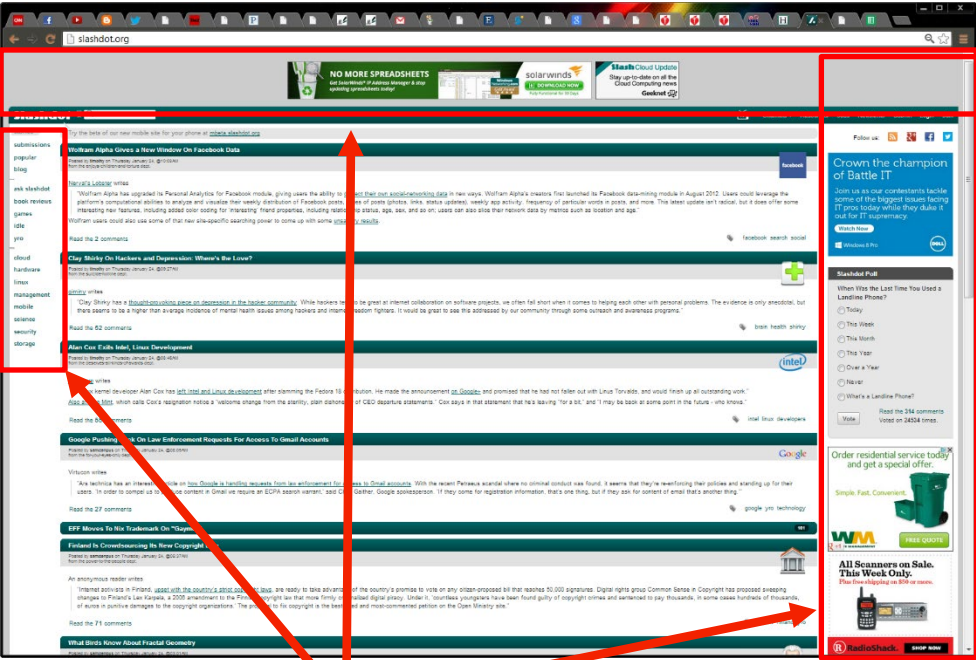
Removing noise

Information Retrieval

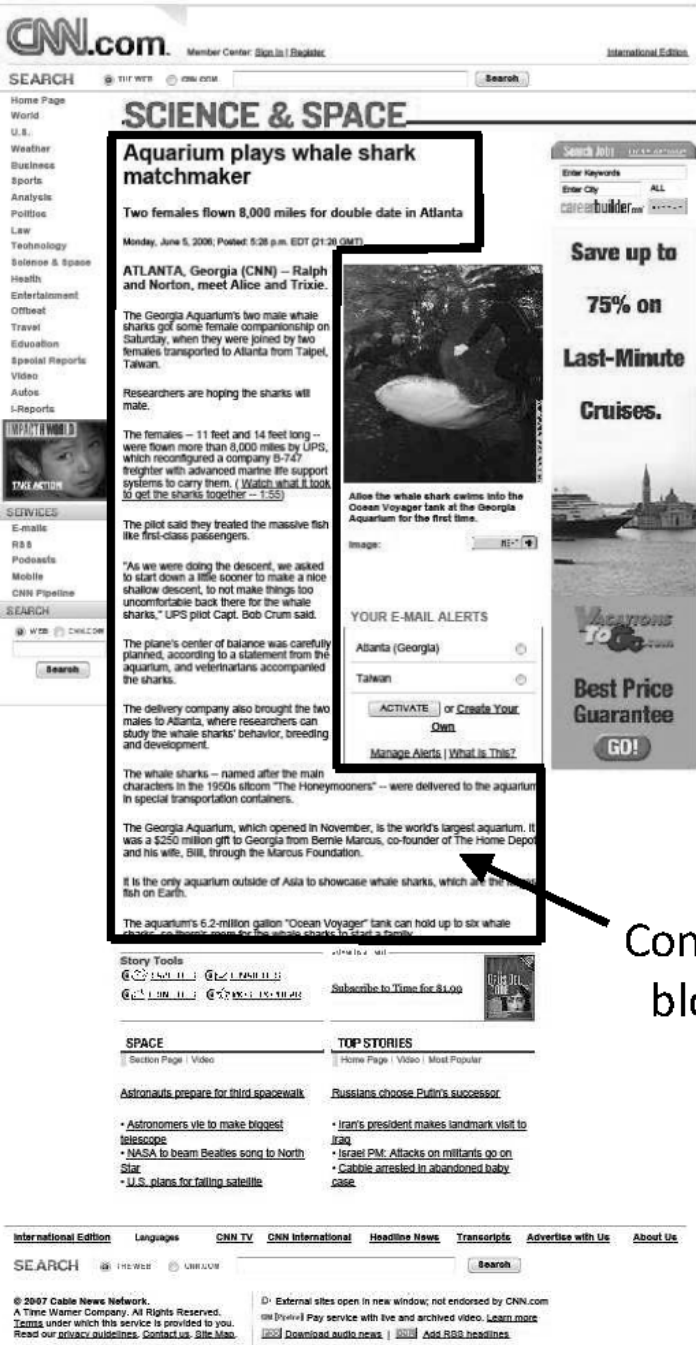
Removing Noise

- Many web pages contain text, links, and pictures that are not directly related to the main content of the page
- This additional material is mostly *noise* that could negatively affect the ranking of the page
- Techniques have been developed to detect the content blocks in a web page
 - Non-content material is either ignored or reduced in importance in the indexing process

Noise Example

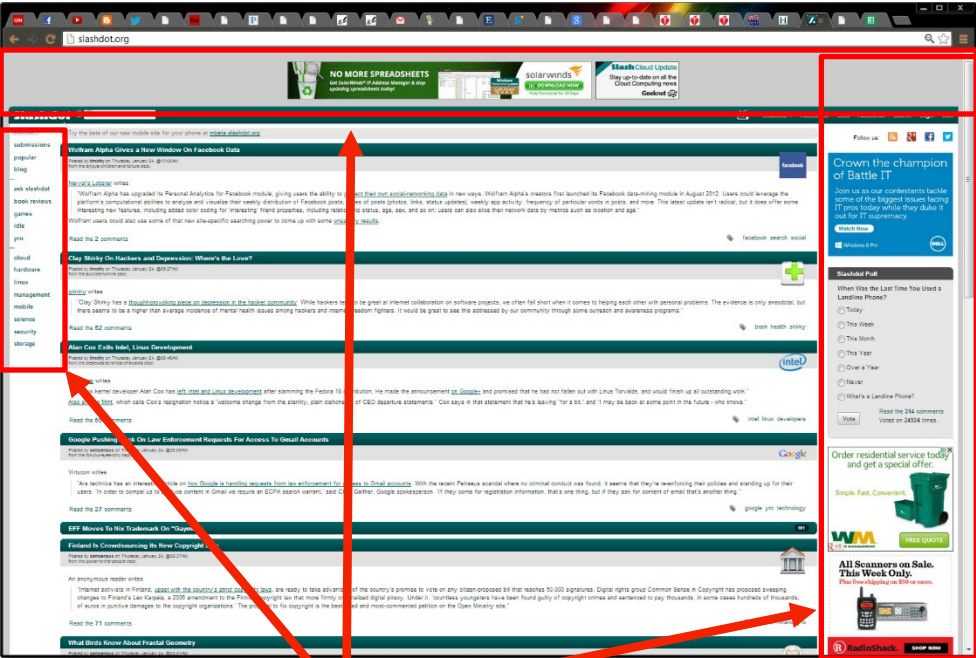


Noise blocks



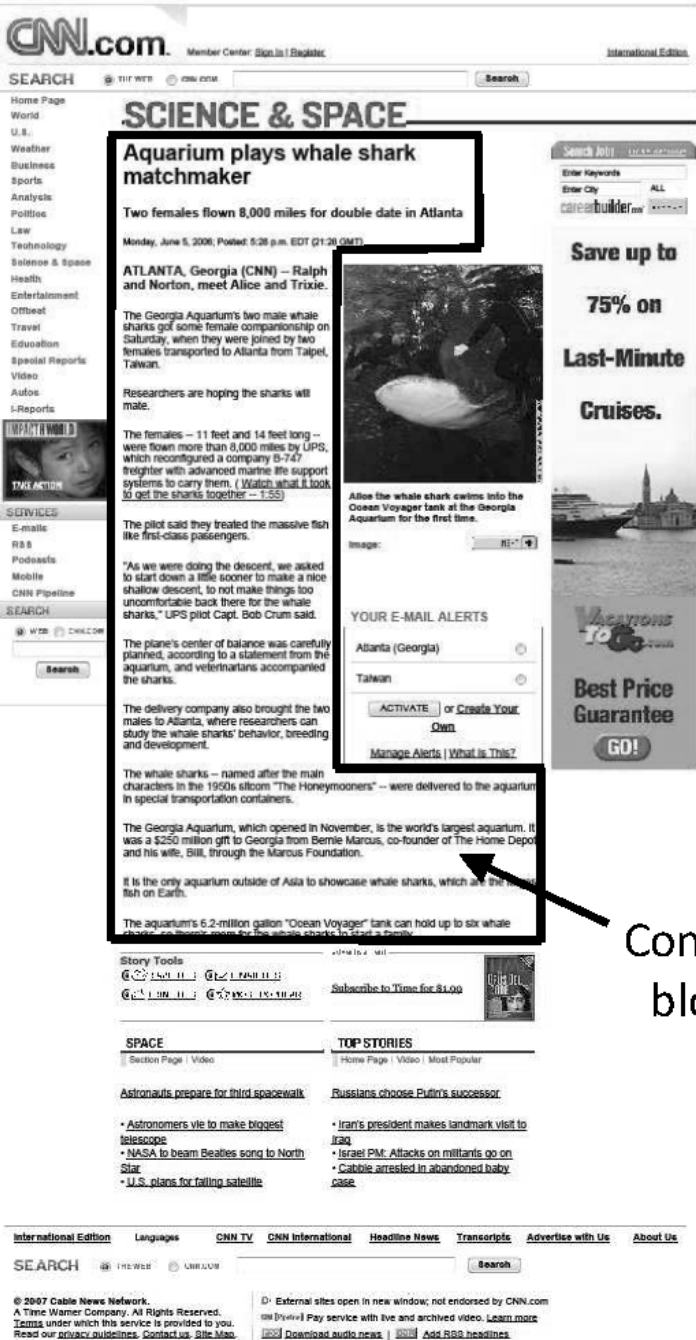
Content block

Noise Example



Noise blocks

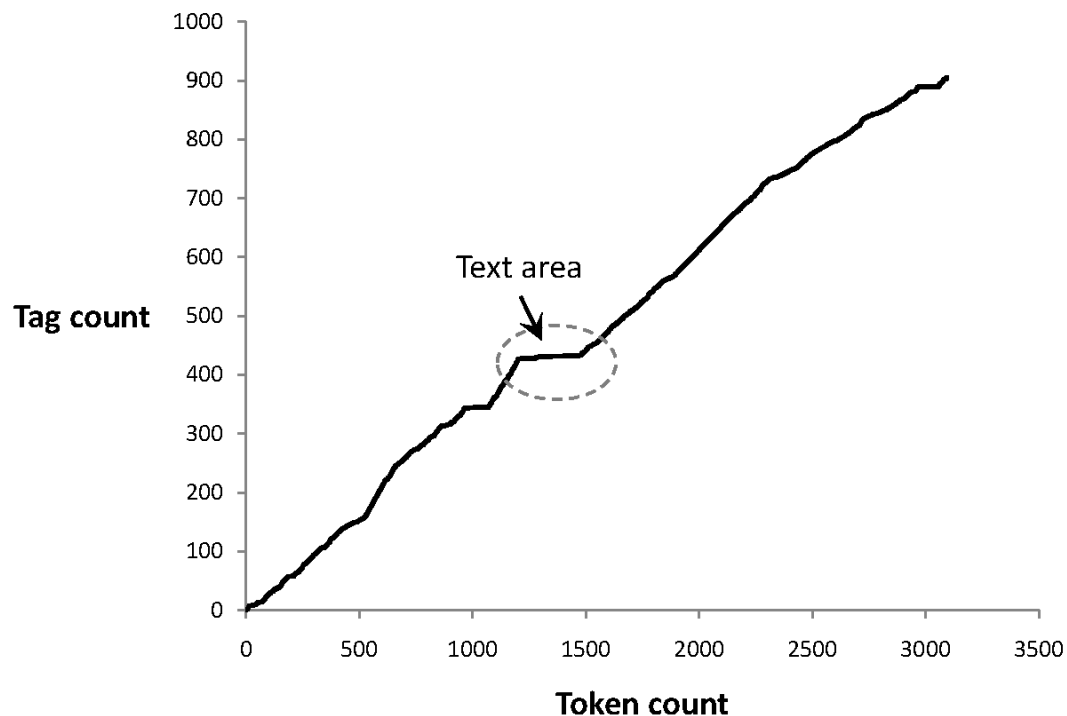
So how can you detect content?



Content block

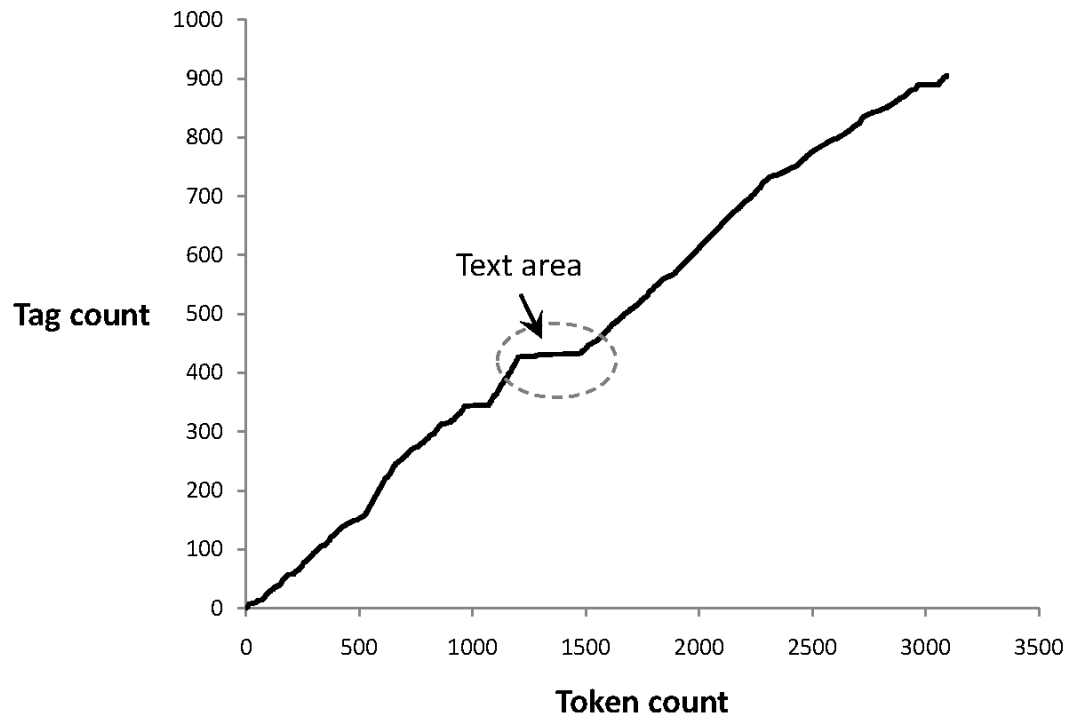
Finding Content Blocks

- Cumulative distribution of tags
 - Document slope curve (e.g. Finn, Kushmerick & Smyth, 2001)



Finding Content Blocks

- Cumulative distribution of tags



- Main text content of the page corresponds to the “plateau” in the middle of the distribution

Finding Content Blocks

- Represent a **web page** as a **sequence of bits**, where $b_n = 1$ indicates that the **n th token is a tag**

Finding Content Blocks

- Represent a **web page** as a **sequence** of **bits**, where $b_n = 1$ indicates that the n th token is a tag
- Optimization problem where we **find values of i and j to maximize** both the number of **tags below i and above j** and the number of **non-tag tokens between i and j**

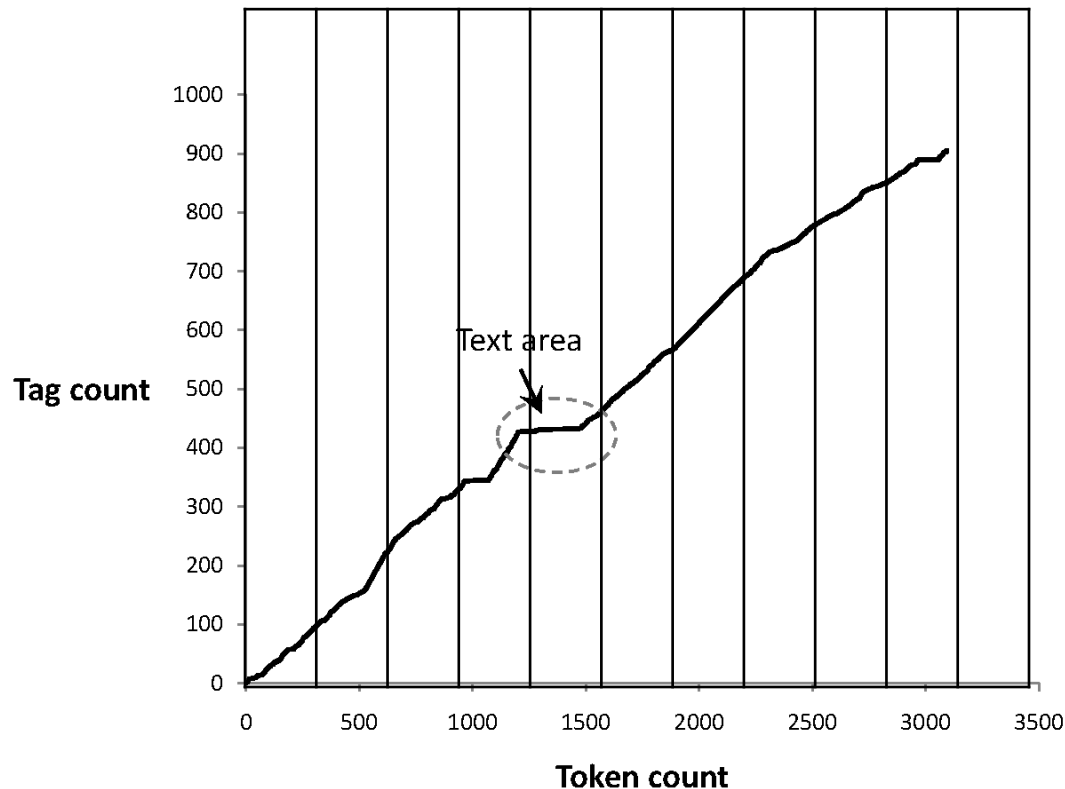
Finding Content Blocks

- Represent a **web page** as a **sequence** of **bits**, where $b_n = 1$ indicates that the n th token is a tag
- Optimization problem where we **find values of i and j to maximize** both the number of **tags below i and above j** and the number of **non-tag tokens between i and j**
- i.e., **find i and j that maximize**

$$\sum_{n=0}^{i-1} b_n + \sum_{n=i}^j (1 - b_n) + \sum_{n=j+1}^{N-1} b_n$$

Finding Content Blocks : Alternative

- Cumulative distribution of tags in the example web page



- Determine the slope of the lines inside slices and iterate, reducing the slice size until you find zero slope windows.

Finding Content Blocks

- Other approaches use Document Object Model (DOM) structure and visual (layout) features
- HTML parser:
 - HTML -> Document Object Model representation
 - Tree like structure

